

Primit 2

Security Assessment

CertiK Assessed on Sept 3rd, 2026





Certik Assessed on Sept 3rd, 2026

Primit 2

The security assessment was prepared by Certik.

Executive Summary

TYPES

DeFi

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 08/20/2026

Final report published on 09/03/2026

Vulnerability Summary



25

Total Findings

18

Resolved

0

Partially Resolved

7

Acknowledged

0

Declined

2 Centralization

2 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

5 Medium

4 Resolved, 1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

12 Minor

11 Resolved, 1 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

6 Informational

3 Resolved, 3 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | PRIMIT 2

Audit Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[Privileged Functions](#)

[External Dependencies](#)

Findings

[PRI-01 : Centralization Related Risks](#)

[PRI-02 : Centralized Control Of Contract Upgrade](#)

[PRI-03 : Unprotected Upgradeable Contract](#)

[PRI-04 : Fund Risks Caused By Insufficient Funds And `backendSigner`](#)

[PRI-05 : Missing `unpause\(\)` Leaves PLP Permanently Pausable-Only](#)

[PRI-06 : `reinitializeSettlementCaps` Missing `onlyOwner` Modifier](#)

[PRI-07 : Negative `remaining` Clears The User's Entire Shared Vault Balance](#)

[PRI-08 : Missing Tier Initialization Validation Leads To Silent Rebate Disablement](#)

[PRI-09 : Missing Duplicate Recipient Validation Leads To Repeated Rebate Distribution](#)

[PRI-10 : Missing Zero Address Validation](#)

[PRI-11 : Incomplete `initialize\(\)` Forces Mandatory Post-Deploy `reinitializer` Calls For Configuration](#)

[PRI-12 : `minWithdraw` Prevents Users From Fully Draining Their Vault Balance](#)

[PRI-13 : `permitCloseOperator\(\)` Does Not Validate That `expiresAt` Is In The Future](#)

[PRI-14 : Issue When `usedPositionBalanceSettlements` Is Already Consumed In `recordPositionCloseAndSettleFor`](#)

[PRI-15 : Undocumented Calculation Precision In `remainingCollateral`](#)

[PRI-16 : Stale `PositionSnapshot` Has No Expiry Before Liquidation](#)

[PRI-17 : `grantHandlerRole` / `revokeHandlerRole` Incompatible With `ADMIN\\_ROLE` -Only Callers](#)

[PRI-23 : `settlePositionBalance\(\)` Unexpectedly Invalidates Pending Withdraw Signatures](#)

[PRI-26 : Confirmation On Fee And Maintenance Margin](#)

[PRI-18 : Redundant Code Components](#)

PRI-19 : Stale Role Entries Remain In `\_roles` After Last Member Is Revoked

PRI-20 : Concerns Regarding The Vault'S On-Chain State

PRI-22 : Confirmation On Incomplete LiquidityVault

PRI-24 : Confirmation On `\_params.closeId` In `\_recordPositionCloseAndSettleFor()`

PRI-25 : Confirmation On Access Control For `\_settlePositionBalance()` Calls

I Optimizations

PRI-21 : `\_recordPositionCloseFor()` Requires Self-Authorization via `\_permitCloseOperator()`

I Appendix

I Disclaimer

CODEBASE | PRIMIT 2

Repository

<https://github.com/Primit-io/primit-contracts-v2-audit/>

Commit

[55b9fec4b3b553dc3d4b875dbc539ece55bb7e60](#)

[7c98edf932bbbb37c7af385f31dd892df67da19d](#)

[c8253041c00a4f38881d55c0d121bad5842a8026](#)

Audit Scope

The files in scope are listed in the appendix.

APPROACH & METHODS | PRIMIT 2

This audit was conducted for Primit 2 to evaluate the security and correctness of the smart contracts associated with the Primit 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Manual Review and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | PRIMIT 2

Overview

Primit 2 is a collateral vault, rebate, referral, PLP liquidity, and liquidation stack on Avalanche. Contracts are UUPS-upgradeable where noted, and build on OpenZeppelin access-control patterns.

Vault (`Vault.sol`) holds the configured ERC20 token: users deposit and withdraw against internal balances; withdrawals require an off-chain EIP-712 signature verified via `SignatureVerifier`. It optionally uses `IReferralStorage` to bind referral codes on deposit or via `setReferralCode`. It exposes backend-signed position-close audit/settlement paths, liquidation settlement via `LiquidationManager`, protocol-fee transfer, settlement credit caps, pause controls, and whitelists `plpVaultAddress` for `debitFromUser` / `creditToUser`.

LiquidationManager (`LiquidationManager.sol`) is a non-upgradeable `Ownable` contract. It stores backend-signed position snapshots, verifies independently signed oracle prices, checks liquidation conditions on-chain, and calls `Vault.settleLiquidation`. Keepers are owner-allowlisted; snapshot updates rely on backend signatures rather than keeper permissions.

LiquidityVault (`LiquidityVault.sol`, PLP) records tiered PLP shares against users' Vault internal balances. Deposits and withdrawals use EIP-712 signatures and move value through `Vault.debitFromUser` / `creditToUser`. UUPS with `AccessControlUpgradeable`.

BufferPool (`BufferPool.sol`) is a non-upgradeable token buffer; `owner` may call `disburse`.

ReferralStorage (`ReferralStorage.sol`) stores referral codes, owners, trader-code bindings (via `HANDLER_ROLE` for vault/rebate), referrer tiers, tier parameters, and optional per-referrer discount overrides. UUPS with `AccessControlUpgradeable`.

RebateDistributor (`RebateDistributor.sol`) pays rebate claims authorized by EIP-712 and a backend signer. It reads `IVault` and `IReferralStorage`, supports owner-only batch rebate distribution.

shared/ (`Role.sol`, `RoleStore.sol`, `Price.sol`, `Errors.sol`) are in-repo libraries/structs in the audit scope but are not deployed as standalone production contracts on mainnet.

Privileged Functions

In the **Primit 2** project, the admin roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the finding `Centralization Related Risks` and `Centralized Control of Contract Upgrade`.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community.

It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the

`Timelock` contract.

External Dependencies

Third-party libraries used in scope:

- `@openzeppelin/contracts`
- `@openzeppelin/contracts-upgradeable`

Configurable or external addresses by contract:

Vault: collateral token (`usdc`), `backendSigner` , `liquidationManager` , optional `referralStorage` , `plpVaultAddress` , `protocolFeeRecipient` , and `owner` .

LiquidationManager: immutable `vault` , `backendSigner` , `oracleSigner` , `settlementRecipient` , keeper allowlist (`setKeeper`), and `owner` .

LiquidityVault: collateral token (`usdc`), `vault` , `ADMIN_ROLE` , `OPERATOR_ROLE` , `SIGNER_ROLE` , and `DEFAULT_ADMIN_ROLE` .

BufferPool: collateral token (`usdc`) and `owner` .

RebateDistributor: collateral token (`usdc`), `vault` , `backendSigner` , optional `referralStorage` , and `owner` .

ReferralStorage: `ADMIN_ROLE` , `HANDLER_ROLE` , and `DEFAULT_ADMIN_ROLE` .

It is assumed that these contracts or addresses are trusted and implemented properly within the whole project. Per the audit inventory, market pair symbols (e.g. `BTCUSDT` , `ETHUSDT`) are opaque identifiers and should not be read as references to a specific stablecoin ticker; the collateral token address configured in each contract is the sole on-chain token assumption.

FINDINGS | PRIMIT 2



25

Total Findings

0

Critical

2

Centralization

0

Major

5

Medium

12

Minor

6

Informational

This report has been prepared for Primit 2 to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 25 issues were identified. Leveraging a combination of Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
PRI-01	Centralization Related Risks	centralization	Centralization	● Acknowledged
PRI-02	Centralized Control Of Contract Upgrade	Centralization	Centralization	● Acknowledged
PRI-03	Unprotected Upgradeable Contract	Logical Issue	Medium	● Resolved
PRI-04	Fund Risks Caused By Insufficient Funds And <code>backendSigner</code>	Denial of Service, Design Issue	Medium	● Acknowledged
PRI-05	Missing <code>unpause()</code> Leaves PLP Permanently Pausable-Only	Logical Issue	Medium	● Resolved
PRI-06	<code>reinitializeSettlementCaps</code> Missing <code>onlyOwner</code> Modifier	Logical Issue	Medium	● Resolved
PRI-07	Negative <code>remaining</code> Clears The User's Entire Shared Vault Balance	Design Issue	Medium	● Resolved
PRI-08	Missing Tier Initialization Validation Leads To Silent Rebate Disablement	Design Issue	Minor	● Resolved
PRI-09	Missing Duplicate Recipient Validation Leads To Repeated Rebate Distribution	Coding Issue	Minor	● Resolved
PRI-10	Missing Zero Address Validation	Code Optimization	Minor	● Resolved

ID	Title	Category	Severity	Status
PRI-11	Incomplete <code>initialize()</code> Forces Mandatory Post-Deploy <code>reinitializer</code> Calls For Configuration	Logical Issue	Minor	● Resolved
PRI-12	<code>minWithdraw</code> Prevents Users From Fully Draining Their Vault Balance	Logical Issue	Minor	● Resolved
PRI-13	<code>permitCloseOperator()</code> Does Not Validate That <code>expiresAt</code> Is In The Future	Logical Issue	Minor	● Resolved
PRI-14	Issue When <code>usedPositionBalanceSettlements</code> Is Already Consumed In <code>recordPositionCloseAndSettleFor</code>	Logical Issue	Minor	● Resolved
PRI-15	Undocumented Calculation Precision In <code>_remainingCollateral</code>	Inconsistency	Minor	● Resolved
PRI-16	Stale <code>PositionSnapshot</code> Has No Expiry Before Liquidation	Logical Issue	Minor	● Resolved
PRI-17	<code>grantHandlerRole</code> / <code>revokeHandlerRole</code> Incompatible With <code>ADMIN_ROLE</code> -Only Callers	Inconsistency	Minor	● Resolved
PRI-23	<code>settlePositionBalance()</code> Unexpectedly Invalidates Pending Withdraw Signatures	Logical Issue	Minor	● Acknowledged
PRI-26	Confirmation On Fee And Maintenance Margin	Logical Issue	Minor	● Resolved
PRI-18	Redundant Code Components	Volatile Code	Informational	● Resolved
PRI-19	Stale Role Entries Remain In <code>_roles</code> After Last Member Is Revoked	Design Issue	Informational	● Resolved
PRI-20	Concerns Regarding The Vault'S On-Chain State	Operational Risk	Informational	● Resolved
PRI-22	Confirmation On Incomplete LiquidityVault	Design Issue	Informational	● Acknowledged

ID	Title	Category	Severity	Status
PRI-24	Confirmation On <code>params.closeId</code> In <code>recordPositionCloseAndSettleFor()</code>	Design Issue	Informational	● Acknowledged
PRI-25	Confirmation On Access Control For <code>_settlePositionBalance()</code> Calls	Design Issue	Informational	● Acknowledged

PRI-01 | Centralization Related Risks

Category	Severity	Location	Status
centralization	● Centralization		● Acknowledged

Description

BufferPool.sol

In the contract `BufferPool`, the role `owner` has authority over the following functions:

- `disburse()`

Any compromise to the `owner` account may allow an attacker to transfer any tokens held by the BufferPool to an arbitrary address.

LiquidityVault.sol

In the contract `LiquidityVault`, the role `ADMIN_ROLE` has authority over the following functions:

- `pause()`

Any compromise to the `ADMIN_ROLE` account may allow an attacker to pause the PLP vault and halt normal user deposit and withdrawal flows.

In the contract `LiquidityVault`, the role `OPERATOR_ROLE` has authority over the following functions:

- `applyFeeShare()`
- `applyMarketPnL()`

Any compromise to the `OPERATOR_ROLE` account may allow an attacker to arbitrarily adjust PLP accounting by inflating or reducing `totalAssets`, which can distort user share value and settlement outcomes.

In the contract `LiquidityVault`, the role `SIGNER_ROLE` is the trusted EIP-712 signing identity for PLP entry and exit. On-chain, `deposit()` and `withdraw()` recover the signer from each user-submitted signature and require `hasRole(SIGNER_ROLE, signer)`:

- `deposit()` — validates signatures over `PLPDeposit(user, amount, tier, nonce, deadline)`
- `withdraw()` — validates signatures over `PLPWithdraw(user, shares, tier, nonce, deadline)`

Any compromise to the private key held by a `SIGNER_ROLE` account may allow an attacker to forge backend authorization for arbitrary PLP deposits and withdrawals, bypassing intended off-chain approval controls even if on-chain role membership is unchanged.

In the contract `LiquidityVault`, the role `DEFAULT_ADMIN_ROLE` (inherited from `AccessControlUpgradeable`) has authority over the following functions:

- `grantRole()`
- `revokeRole()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke `ADMIN_ROLE`, `OPERATOR_ROLE`, and `SIGNER_ROLE`, thereby taking over pause, accounting, and PLP signature verification authority.

In the contract `LiquidityVault`, the role `ADMIN_ROLE` (inherited from `UUPSUpgradeable`) has authority over the following functions:

- `upgradeToAndCall()`

Any compromise to the `ADMIN_ROLE` account may allow an attacker to upgrade the proxy to a malicious implementation and take full control of the contract.

RebateDistributor.sol

In the contract `RebateDistributor`, the role `owner` has authority over the following functions:

- `reinitializeEip712DomainVersion()`
- `batchSyncRebates()`
- `setBackendSigner()`
- `setReferralStorage()`

Any compromise to the `owner` account may allow an attacker to rotate the rebate signer, redirect referral storage dependencies, modify the EIP-712 domain version, and distribute the contract's tokens to arbitrary addresses via batch rebate sync.

In the contract `RebateDistributor`, the role `backendSigner` is the trusted EIP-712 signing key for rebate claims. `claimRebate()` verifies a backend signature over the claim payload and increments `rebateNonces[msg.sender]` before payout:

- `claimRebate()`

Any compromise to the `backendSigner` private key may allow an attacker to authorize arbitrary or inflated rebate claims and drain tokens through forged EIP-712 signatures, independent of intended off-chain rebate accounting.

In the contract `RebateDistributor`, the role `owner` (inherited from `UUPSUpgradeable`) has authority over the following functions:

- `upgradeToAndCall()`

Any compromise to the `owner` account may allow an attacker to upgrade the proxy to a malicious implementation and take full control of the contract.

In the contract `RebateDistributor`, the role `owner` (inherited from `OwnableUpgradeable`) has authority over the following functions:

- `transferOwnership()`
- `renounceOwnership()`

Any compromise to the `owner` account may allow an attacker to transfer ownership to another address or renounce ownership.

ReferralStorage.sol

In the contract `ReferralStorage`, the role `HANDLER_ROLE` has authority over the following functions:

- `setTraderReferralCode()`

Any compromise to the `HANDLER_ROLE` account may allow an attacker to bind traders to arbitrary referral codes, affecting referral attribution and downstream rebate distribution.

In the contract `ReferralStorage`, the role `ADMIN_ROLE` has authority over the following functions:

- `setTier()`
- `setReferrerTier()`
- `setReferrerDiscountShare()`
- `grantHandlerRole()`
- `revokeHandlerRole()`

Any compromise to the `ADMIN_ROLE` account may allow an attacker to reconfigure referral economics, arbitrarily assign referrer tiers and discount shares, and add or remove handler contracts.

In the contract `ReferralStorage`, the role `DEFAULT_ADMIN_ROLE` has authority over the following functions:

- `govSetCodeOwner()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to reassign ownership of any referral code.

In the contract `ReferralStorage`, the role `DEFAULT_ADMIN_ROLE` (inherited from `AccessControlUpgradeable`) has authority over the following functions:

- `grantRole()`
- `revokeRole()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to grant or revoke `DEFAULT_ADMIN_ROLE`, `ADMIN_ROLE`, and `HANDLER_ROLE`, thereby taking over the referral system.

In the contract `ReferralStorage`, the role `DEFAULT_ADMIN_ROLE` (inherited from `UUPSUpgradeable`) has authority over the following functions:

- `upgradeToAndCall()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow an attacker to upgrade the proxy to a malicious implementation and take full control of the contract.

RoleStore.sol

In the contract `RoleStore`, the role `Role.ADMIN` has authority over the following functions:

- `grantRole()`
- `revokeRole()`

Any compromise to the `Role.ADMIN` account may allow an attacker to arbitrarily grant or revoke protocol-wide roles, including administrative, liquidation, snapshot, oracle, and other keeper authorities.

LiquidationManager.sol

In the contract `LiquidationManager`, the role `owner` has authority over the following functions:

- `setBackendSigner()`
- `setOracleSigner()`
- `setSettlementRecipient()`
- `setKeeper()`

Any compromise to the `owner` account may allow an attacker to replace the liquidation signer set, redirect settlement proceeds, and appoint malicious keepers.

In the contract `LiquidationManager`, the role `backendSigner` is the trusted `eth_sign` signing key for on-chain position snapshots. Anyone may relay valid payloads, but `updateSnapshot()` and `closeSnapshot()` only succeed when the recovered signer equals `backendSigner`:

- `updateSnapshot()` — consumes backend-signed `PositionSnapshot` payloads
- `closeSnapshot()` — consumes backend-signed close digests

Any compromise to the `backendSigner` private key may allow an attacker to publish or close arbitrary position snapshots, fabricating liquidation-eligible on-chain state.

In the contract `LiquidationManager`, the role `oracleSigner` is the trusted `eth_sign` signing key for mark prices used during liquidation. `liquidate()` verifies the oracle payload signature against `oracleSigner` before computing margin and debiting the Vault:

- oracle price signatures consumed inside `liquidate()`

Any compromise to the `oracleSigner` private key may allow an attacker to authorize manipulated mark prices and induce wrongful liquidations whenever a keeper submits a matching oracle payload with a valid snapshot.

In the contract `LiquidationManager`, the role `keeper` has authority over the following functions:

- `liquidate()`

Any compromise to a `keeper` account may allow an attacker to trigger liquidation settlement whenever valid backend-signed snapshot data and valid oracle-signed price data are available, redirecting proceeds to `settlementRecipient`.

In the contract `LiquidationManager`, the role `owner` (inherited from `Ownable`) has authority over the following functions:

- `transferOwnership()`
- `renounceOwnership()`

Any compromise to the `owner` account may allow an attacker to transfer ownership to another address or renounce ownership.

Vault.sol

In the contract `Vault`, the role `owner` has authority over the following functions:

- `reinitializeEip712DomainVersion()`
- `setLiquidationManager()`
- `setDailySettlementCreditCap()`
- `setDailyUserSettlementCreditCap()`
- `setBackendSigner()`
- `setReferralStorage()`
- `setDepositPaused()`
- `setWithdrawPaused()`
- `reinitializeAddPLPAddress()`
- `reinitializeProtocolFees()`
- `setProtocolFeeRecipient()`
- `pause()`
- `unpause()`
- `emergencyWithdraw()`
- `setMinDeposit()`
- `setMinWithdraw()`

Any compromise to the `owner` account may allow an attacker to reconfigure core dependencies and settlement controls, pause or resume the system, redirect protocol fees, authorize a PLP bridge address, change minimum thresholds, and withdraw tokens from the contract during an emergency pause.

In the contract `Vault`, the role `backendSigner` is the primary trusted EIP-712 signing key for user fund movement and balance settlement authorization:

- `withdraw()` — verifies `Withdraw(user, amount, nonce, deadline)`
- `recordPositionClose()` — verifies backend-signed position-close audit payloads
- `recordPositionCloseFor()` — verifies backend-signed close payloads relayed by approved operators

- `recordPositionCloseAndSettleFor()` — verifies backend-signed close-and-settle payloads, including `balanceDelta` and fees
- `settlePositionBalance()` — may be called directly by `backendSigner` to apply signed balance deltas

Any compromise to the `backendSigner` private key may allow an attacker to authorize arbitrary withdrawals, fabricate close records, and directly credit or debit user Vault balances through forged EIP-712 signatures or direct settlement calls. Users cannot withdraw without a valid signature from this key, so its availability and integrity are a single point of trust for exit liquidity.

In the contract `Vault`, the role `liquidationManager` has authority over the following functions:

- `settleLiquidation()`
- `settlePositionBalance()`

Any compromise to the `liquidationManager` account may allow an attacker to debit user balances, clear accounts, and transfer liquidation proceeds to arbitrary recipients configured by the manager contract.

In the contract `Vault`, the role `plpVaultAddress` has authority over the following functions:

- `debitFromUser()`
- `creditToUser()`

Any compromise to the `plpVaultAddress` account may allow an attacker to arbitrarily debit or credit user Vault balances through the PLP bridge.

In the contract `Vault`, the role `owner` (inherited from `UUPSUpgradeable`) has authority over the following functions:

- `upgradeToAndCall()`

Any compromise to the `owner` account may allow an attacker to upgrade the proxy to a malicious implementation and take full control of the contract.

In the contract `Vault`, the role `owner` (inherited from `OwnableUpgradeable`) has authority over the following functions:

- `transferOwnership()`
- `renounceOwnership()`

Any compromise to the `owner` account may allow an attacker to transfer ownership to another address or renounce ownership.

During the remediation process, the following centralization-related functions were added:

LiquidityVault.sol

In the contract `LiquidityVault`, the role `ADMIN_ROLE` has authority over the following functions:

- `unpause()`

Any compromise to the `ADMIN_ROLE` account may allow an attacker to resume PLP deposits and withdrawals after a pause.

LiquidationManager.sol

In the contract `LiquidationManager`, the role `owner` has authority over the following functions:

- `setMaxSnapshotAge()`
- `setMaxLiquidationFeeRate()`
- `setMaintenanceMarginRateBounds()`

Any compromise to the `owner` account may allow an attacker to retune snapshot staleness limits, change maximum liquidation fee rate, and adjust maintenance-margin bounds used during liquidation checks.

Vault.sol

In the contract `Vault`, the role `owner` has authority over the following functions:

- `reinitializeSettlementCaps()`
- `setPlpVaultAddress()`
- `setMaxProtocolFeeAmount()`

Any compromise to the `owner` account may allow an attacker to re-initialize settlement credit caps, rotate the whitelisted PLP bridge address, and change the per-settlement protocol-fee ceiling.

In the contract `Vault`, the role `backendSigner` has authority over the following functions:

- `withdrawAll()`

Any compromise to the `backendSigner` account may allow an attacker to authorize full-balance withdrawals via a signed `withdrawAll` message.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;

AND

- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;

AND

- Introduction of a DAO/governance/voting module to increase transparency and user involvement.

AND

- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.

OR

- Remove the risky functionality.

I Alleviation

[Primit 2, 08/21/2026]: Status: Acknowledged.

We acknowledge CertiK's enumeration of privileged roles across the seven contracts (BufferPool, LiquidityVault, RebateDistributor, ReferralStorage, RoleStore, LiquidationManager, Vault) and confirm the described attack surface is accurate as of the audited commit 55b9fec4b3b553dc3d4b875dbc539ece55bb7e60.

We are proceeding with CertiK's recommended short-term remediation Gnosis Safe (3-of-5 multi-signature) plus TimelockController (48-hour delay) across all seven privileged roles with estimated delivery within four weeks of this response. Concrete scheduled steps:

(1) deploy Gnosis Safe on Avalanche C-Chain with five signers;

(2) deploy TimelockController with proposer=Safe, executor=Safe, delay=48h;

(3) rotate owner / DEFAULT_ADMIN_ROLE on all seven in-scope contracts to the Timelock, retaining a separate PAUSE_GUARDIAN role held by the Safe directly (bypassing Timelock) for incident response;

(4) rotate the five signer accounts (backendSigner instances on Vault / RebateDistributor / LiquidationManager, plus oracleSigner and SIGNER_ROLE) to Safe-controlled hot wallets, per our prior architecture decision record (ADR-2026-0428-002);

(5) publish a public disclosure post listing the Timelock address, Safe address, obfuscated signer roster, and upgrade

governance flow.

Once the migration is complete, we will update this finding to Partially Resolved and share the deployed Timelock address, Gnosis Safe address, and the accompanying medium/blog link. A DAO/governance module is on our long-term roadmap and will be introduced after the protocol matures and community stewardship is in place; we do not intend to renounce ownership permanently, as the ability to upgrade the contracts is required to respond to future audit findings.

PRI-02 | Centralized Control Of Contract Upgrade

Category	Severity	Location	Status
Centralization	● Centralization	plp_contract/src/LiquidityVault.sol (init-0812): 124; rebate_contract/src/contracts/core/referral/RebateDistributor.sol (init-0812): 371~373; referral_storage_contract/src/contracts/referral/ReferralStorage.sol (init-0812): 292~294; vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 1006~1008	● Acknowledged

Description

In the contract `LiquidityVault`, the role `ADMIN_ROLE` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `ADMIN_ROLE` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

In the contract `RebateDistributor`, the role `owner` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `owner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

In the contract `ReferralStorage`, the role `DEFAULT_ADMIN_ROLE` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

In the contract `Vault`, the role `owner` has the authority to update the implementation contract behind the proxy contract.

Any compromise to the `owner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (2/3, 3/5) wallet can be used to prevent a single point of failure due to a private key

compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
- OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

I Alleviation

[Primit 2, 08/21/2026]: Status: Acknowledged.

We acknowledge that the upgrade authority on the four in-scope proxy contracts (LiquidityVault ADMIN_ROLE, RebateDistributor owner, ReferralStorage DEFAULT_ADMIN_ROLE, Vault owner) is currently held by an EOA-controlled account, and that this represents a single point of failure.

This finding is a subset of the broader centralization concern already addressed in our response to PRI-01: the same Gnosis Safe (3-of-5 multi-signature) plus TimelockController (48-hour delay) governance stack scheduled for delivery within four weeks will take ownership of these four proxies, gating every future upgradeToAndCall call through the 48-hour Timelock queue and requiring multi-signature approval on the Safe.

Concrete additional steps specific to this finding:

- (1) after Timelock deployment, transfer or grant the upgrade authority on each of the four proxies (LiquidityVault ADMIN_ROLE, RebateDistributor owner, ReferralStorage DEFAULT_ADMIN_ROLE, Vault owner) to the Timelock address;
- (2) revoke the previous EOA holder from the upgrade role in the same transaction batch;
- (3) advance-notify the community via the same medium/blog post prior to any subsequent implementation migration. Once the migration is complete, we will update this finding to Partially Resolved and share the deployed Timelock address, Gnosis Safe address, and the accompanying medium/blog link. As with PRI-01, a DAO/governance module for upgrade control is on our long-term roadmap;

we do not intend to renounce upgrade ownership permanently, as the ability to migrate implementations is required to respond to future audit findings and to ship the remaining fixes listed in this same report.

PRI-03 | Unprotected Upgradeable Contract

Category	Severity	Location	Status
Logical Issue	● Medium	plp_contract/src/LiquidityVault.sol (init-0812): 100	● Resolved

Description

The `LiquidityVault` logic contract does not protect the initializer. An attacker can front-run the `initialize` call and assume ownership of the logic contract. Once in control, the attacker can perform privileged operations, misleading users into believing that they are interacting with the legitimate owner of the upgradeable contract.

Recommendation

We recommend adding

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() initializer {...}
```

OR

```
/// @custom:oz-upgrades-unsafe-allow constructor
constructor() {
    ...
    _disableInitializers();
}
```

This addition will prevent the function `revert()` from being called directly in the implementation contract, but the proxy will still be able to initialize its storage variables.

Alleviation

[Primit 2, 08/27/2026]: The team heeded the advice and resolved the issue by adding `_disableInitializers()` in `contstructor()` in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-04 | Fund Risks Caused By Insufficient Funds And backendSigner

Category	Severity	Location	Status
Denial of Service, Design Issue	● Medium	vault_contract/src/contracts/core/vault/Vault.sol (init-081 2): 305~309, 472~483, 762~769, 979~986	● Acknowledged

Description

In the Vault, `emergencyWithdraw()` allows all funds in the Vault to be withdrawn after the contract is paused, introducing a centralized fund withdrawal risk.

`Vault.sol`

```

979     function emergencyWithdraw(address to, uint256 amount) external
onlyOwner {
980         if (!paused()) {
981             revert("Not paused");
982         }
983         if (to == address(0)) revert ZeroAddress();
984         usdc.safeTransfer(to, amount);
985         emit EmergencyWithdraw(to, amount);
986     }

```

`withdraw()` requires a signature from the `backendSigner`. If a user is unable to obtain the required signature or the `backendSigner` becomes unavailable, the user will be unable to withdraw their deposited funds.

`Vault.sol`

```

305         if (!SignatureVerifier.verifyWithdrawSignature(
306             DOMAIN_SEPARATOR, msg.sender, amount, nonce, deadline,
signature, backendSigner
307         )) {
308             revert InvalidSignature();
309         }

```

In addition, `settlePositionBalance()` allows the `liquidationManager` (although there is currently no corresponding call implemented in `LiquidationManager`) and the `backendSigner` to modify users' deposited balances without any corresponding token inflow or outflow.

`Vault.sol`

```

472     function settlePositionBalance(address user, int256 balanceDelta, bytes32
settlementKey)
473         external
474         override
475         nonReentrant
476         returns (uint256 creditedAmount, uint256 debitedAmount)
477     {
478         if (msg.sender != liquidationManager && msg.sender != backendSigner) {
479             revert UnauthorizedLiquidationManager(msg.sender);
480         }
481
482         return _settlePositionBalance(user, balanceDelta, settlementKey);
483     }

```

`recordPositionCloseAndSettleFor()` allows an unexpired Close Operator, with a signature from the `backendSigner`, to invoke the same `_settlePositionBalance()` logic used by `settlePositionBalance()` to modify a user's balance, likewise without any token inflow or outflow.

`Vault.sol`

```

762     function recordPositionCloseAndSettleFor(PositionCloseSettlementParams
calldata params)
763         external
764         override
765         nonReentrant
766         returns (uint256 creditedAmount, uint256 debitedAmount)
767     {
768         return _recordPositionCloseAndSettleFor(params);
769     }
770
771     function _recordPositionCloseAndSettleFor(PositionCloseSettlementParams
calldata params)
772         internal
773         returns (uint256 creditedAmount, uint256 debitedAmount)
774     {
775         if (params.user == address(0)) revert ZeroAddress();
776         if (block.timestamp > params.deadline) {
777             revert SignatureExpired(params.deadline, block.timestamp);
778         }
779
780         uint64 expiresAt = closeOperatorApprovalExpiries[params.user][msg.
sender];
781         if (expiresAt < block.timestamp) {
782             revert CloseOperatorApprovalExpired(params.user, msg.sender,
expiresAt, block.timestamp);
783         }

```

The security of these two functions depends on the out-of-scope `backendSigner` and the project team's operational processes. For the purposes of this review, we treat these components as black boxes and assume that they invoke the relevant functions to modify balances and issue the corresponding signatures as intended by the project team. Additionally,

increases or decreases in `balance` do not result in corresponding token transfers into or out of the contract. We would like to confirm the intended design and logic behind this behavior.

In addition, the `recordPositionClose()`, and `recordPositionCloseFor()` functions serve only as record-keeping mechanisms in this Vault, while the `recordPositionClose()` function can also be replayed multiple times. For the purposes of this audit, we assume that these functions are invoked securely and as intended by the upstream components. At the same time, we would like to confirm whether it is intended for users to be able to replay `recordPositionClose()` multiple times and whether such repeated calls could affect the functionality of the system.

Recommendation

Regarding user withdrawals, the protocol team should ensure that the contracts and Vault always maintain sufficient funds to satisfy user withdrawals and that users are not prevented from withdrawing their deposited funds due to signature-related issues.

Regarding the balance-modifying functions `recordPositionCloseAndSettleFor()` and `settlePositionBalance()`, we would like to confirm the rationale for modifying users' balances without any corresponding token transfers. For the purposes of this audit, these mechanisms are treated as a black box, and we assume that the `backendSigner` issues signatures and invokes the relevant functions correctly and in accordance with the protocol's intended behavior.

We assume that the upstream components use and invoke these record functions securely and as intended. At the same time, we would like to confirm whether allowing `recordPositionClose` to be replayed multiple times is part of the intended design.

Alleviation

[Primit 2, 08/21/2026]: Status: Acknowledged.

This finding bundles four distinct sub-issues; we agree with the enumeration and address each below.

(1) `emergencyWithdraw()`: the concern is a specific instance of the general privileged-role centralization we already track under PRI-01 and PRI-02. We are keeping `emergencyWithdraw()` as a catastrophic escape hatch (critical bug, oracle failure, chain incident) but the owner role that gates it is being migrated to a Gnosis Safe (3-of-5 multi-signature) plus TimelockController (48-hour delay), so any invocation of `emergencyWithdraw()` after the migration requires Safe quorum and 48-hour on-chain notice, with the pause/unpause path kept as a separate `PAUSE_GUARDIAN` role held by the Safe directly (Safe-only, no timelock) for incident response. Concrete progress under PRI-01/02: target four-week delivery, with the deployed Timelock and Safe addresses shared once the migration is complete.

(2) `withdraw()` dependency on `backendSigner`: we acknowledge the liveness single-point-of-failure. The signature check is intended (it enforces a strict backend-side authorization cap on how much a user can withdraw at any moment, so that a user cannot front-run pending settlement or liquidation with a raw withdraw call), but signer unavailability would strand user funds. Current mitigations: (a) `emergencyWithdraw()` lets the owner rescue funds under a pause during a signer outage, (b) `permitCloseOperator()` lets a user delegate close-and-settle to an alternate operator, reducing dependency on a single hot signer, and (c) the backend signer key is deployed on a Safe-controlled hot wallet with a documented rotation runbook. Long-term hardening on our roadmap: a signer-less exit route that allows a user to submit an on-chain withdraw request during pause, wait a fixed delay window (e.g. 72 hours) with no backend counter-signature, and then self-execute; this will be

tracked as a separate ticket and delivered in a follow-up release, at which point we will move this sub-issue to Partially Resolved.

(3) `settlePositionBalance()` and `recordPositionCloseAndSettleFor()` modifying balances without token flow: this is the intended design of a CEX-style collateral ledger, and mirrors the same accounting model used by GMX v2, dYdX v4, and Hyperliquid. Token transfers only occur on `deposit()` and `withdraw()`; intra-position PnL, funding, and liquidation settlements move the internal `_balances` ledger only, and the pooled collateral in the Vault is sized to always back the aggregate credited balance. Replay is prevented by `usedPositionBalanceSettlements[settlementKey]` on `settlePositionBalance()` and by `usedCloseIds[closeId]` on `recordPositionCloseAndSettleFor()`. Additional on-chain safety rails already in the code: `type(int256).min` is explicitly rejected, negative deltas are truncated to the current chain balance so a loss on a zero-balance account settles cleanly at zero rather than underflowing, and positive deltas pass through `_consumeSettlementCreditCap(user, credit)` which enforces a per-user rate-limit on credited amounts.

(4) `recordPositionClose()` replayability: intended by design. This function is a pure audit-event emitter (only emit `PositionClosed(...)`; no state mutation, no `_balances` write, no `_settlePositionBalance` call), so a replay produces a duplicate off-chain event but has no on-chain effect on user balances or system solvency. The signature carries a deadline which bounds the replay window, and our off-chain indexers already deduplicate by the `(positionId, closedAt)` tuple. `recordPositionCloseFor()` and `recordPositionCloseAndSettleFor()`, which do mutate state, remain replay-protected by `usedCloseIds[closeId]`. This design contract is now explicit in the natspec: see <https://github.com/Primit-io/primit-avax-contracts/pull/6> which adds a dedicated dev-note on `recordPositionClose` stating that duplicate emissions are permitted, that off-chain consumers must deduplicate by `(positionId, closedAt)`, and that callers needing replay-guarded record-and-settle semantics should use `recordPositionCloseFor` or `recordPositionCloseAndSettleFor`.

PRI-05 | Missing `unpause()` Leaves PLP Permanently Pausable-Only

Category	Severity	Location	Status
Logical Issue	● Medium	plp_contract/src/LiquidityVault.sol (init-0812): 208~210	● Resolved

Description

`LiquidityVault` inherits `PausableUpgradeable` and exposes `pause()` to `ADMIN_ROLE`, but does not expose a corresponding `unpause()`. Both `deposit` and `withdraw` are gated by `whenNotPaused`. After `pause()` is called, user deposit and withdrawal paths remain blocked unless the contract is upgraded via UUPS.

Add a symmetric admin endpoint:

```
function unpause() external onlyRole(ADMIN_ROLE) {
    _unpause();
}
```

Recommendation

We recommend adding a corresponding `unpause` function.

Alleviation

[Primit 2, 09/03/2026]: The team heeded the advice and resolved the issue by adding a corresponding `unpause()` function in commit [c8253041c00a4f38881d55c0d121bad5842a8026](#)

PRI-06 | `reinitializeSettlementCaps` Missing `onlyOwner` Modifier

Category	Severity	Location	Status
Logical Issue	● Medium	<code>vault_contract/src/contracts/core/vault/Vault.sol</code> (init-0812): 231~236	● Resolved

Description

`reinitializeSettlementCaps` uses `reinitializer(3)` but lacks `onlyOwner`, while other reinitializers in the same contract are owner-restricted:

`Vault.sol`

```
221     function reinitializeEip712DomainVersion(string memory _domainVersion)
external reinitializer(2) onlyOwner {
```

`Vault.sol`

```
231     function reinitializeSettlementCaps(uint256 globalCap, uint256 perUserCap)
external override reinitializer(3) {
232         dailySettlementCreditCap = globalCap;
233         dailyUserSettlementCreditCap = perUserCap;
```

`Vault.sol`

```
894     function reinitializeProtocolFees(address initialRecipient) external
override reinitializer(4) onlyOwner {
```

Without `onlyOwner`, any address can front-run and set the initial values of `dailySettlementCreditCap` and `dailyUserSettlementCreditCap` while the initialization version is below 3 and `reinitializer(3)` has not yet been executed.

Recommendation

We recommended adding `onlyOwner` to `reinitializeSettlementCaps`, consistent with the other reinitializers in this contract.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding `onlyOwner` to `reinitializeSettlementCaps` in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-07 Negative `remaining` Clears The User's Entire Shared Vault Balance

Category	Severity	Location	Status
Design Issue	● Medium	vault_contract/src/contracts/core/liquidation/LiquidationManager.sol (init-0812): 198~204; vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 450~451	● Resolved

Description

In `liquidate`, any negative `remaining` sets `clearBalance = true`:

`LiquidationManager.sol`

```
198         uint256 requestedDebit = (snap.sizeUsd * snap.liquidationFeeRate) /  
1e18;  
199         bool clearBalance = remaining < 0;  
200         liquidationKey = keccak256(abi.encodePacked(positionId, snap.nonce,  
oraclePrice.price, oraclePrice.updatedAt));  
201  
202         _snapshots[positionId].status = SnapshotStatus.Liquidated;  
203         debitedAmount =  
204             vault.settleLiquidation(snap.user, settlementRecipient,  
requestedDebit, clearBalance, liquidationKey);
```

When `clearBalance` is true, `settleLiquidation` debits the user's full `_balances[user]`, ignoring `requestedAmount`:

`Vault.sol`

```
450         uint256 balance = _balances[user];  
451         debitedAmount = clearBalance || requestedAmount > balance ? balance :  
requestedAmount;
```

`_balances[user]` is a shared Vault balance, not position-isolated collateral. A position with `remaining < 0` can therefore sweep all unrelated funds in that account (deposits, PLP credits, other position backing), not just the liquidated position's deficit or liquidation fee. There is no on-chain cap to the position shortfall and no automatic credit-back.

Recommendation

We recommended capping liquidation debits to a position-scoped obligation (deficit plus configured liquidation fee) instead of clearing the entire shared Vault balance when `remaining < 0`, and recording any uncovered shortfall explicitly as bad debt.

Alleviation

[Primit 2, 08/21/2026]: Status: Resolved.

Fixed in PR <https://github.com/Primit-io/primit-avax-contracts/pull/9> and deployed to Avalanche mainnet at block 93328756. Root fix (LiquidationManager side; Vault ABI unchanged): the liquidation obligation is now computed on-chain as position-scoped, $\text{obligation} = \text{liquidationFee} + \max(0, -\text{remaining})$, and LiquidationManager.liquidate now always passes `clearBalance=false` to Vault.settleLiquidation. Vault-side behaviour on `clearBalance=false` is exactly $\min(\text{obligation}, \text{balance})$, so the user's shared `_balances` is capped to the position's own fee-plus-shortfall obligation and the remainder (unassigned deposits, PLP credits, other open positions' collateral) is preserved. If obligation exceeds the debited amount, the residue is surfaced explicitly through a new event LiquidationBadDebt(bytes32 indexed positionId, bytes32 indexed liquidationKey, address indexed user, uint256 obligation, uint256 debitedAmount, uint256 badDebt) for off-chain bad-debt accounting, exactly as recommended. TDD coverage in the same PR: three new Foundry tests assert that a liquidation with `remaining=-10` and `balance=1000` debits exactly 15 ($\text{fee}=5 + \text{shortfall}=10$) and leaves 985 preserved, that a liquidation whose obligation exceeds the available balance debits only the balance and emits LiquidationBadDebt with the correct obligation and residue, and a dedicated no-full-sweep regression guard asserts the leftover balance stays above 900 ether and the settlement recipient never receives more than ~20 ether. Full vault_contract Foundry suite: 74 of 74 tests pass. Mainnet rollout: because LiquidationManager is a direct (non-proxy) contract, remediation was done by redeploying LiquidationManager and rewiring the Vault via Vault.setLiquidationManager in the same atomic broadcast (Vault upgrade was not required and was not performed). New LiquidationManager address: 0xb3a280809Bc29e010C67B45d49429B7b8fFF27E5. Previous LiquidationManager (now severed from the Vault): 0xceb119aadd56b2fe602b634d50c40af21e985de1. Vault proxy address is unchanged at 0xdf19d902fcb6295366a06620f76dcb49e686c403. Broadcast transactions: LiquidationManager deploy 0x05ef970c30e9502a486a1f368b8d65a22f49ec1955e99cf07cb8dbb57c67f3d0, Vault.setLiquidationManager 0x62bf11b522ffb045fe07f804a5fdfac504213664dcd28f251bbc18f4607ac3b7, LiquidationManager.setKeeper 0x13346b274078bf6e2080c1893096e7e87c98e8edd976f09188bf6f48ca253e79. Source code is verified on Routscan (Snowtrace). Post-upgrade on-chain verification: Vault.liquidationManager() reads the new LM address, and the new LM's vault, backendSigner, oracleSigner, settlementRecipient, owner and keepers(backendSignerAddress) all match the previous manager's parameters (no operational drift). Legacy note: the previous LiquidationManager at 0xceb119... has never emitted a LiquidationExecuted event across the entire V2 mainnet lifetime (3 million blocks / roughly 70 days scanned), so no historical liquidations existed and the manager replacement carried zero legacy-state migration risk.

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by deducting only the liquidation fee and the shortfall when `remaining` is negative, i.e., `liquidationFee + max(0, -remaining)`, instead of clearing the user's entire balance in the vault in commit [7c98edf932bbb37c7af385f31dd892df67da19d](#)

PRI-08 Missing Tier Initialization Validation Leads To Silent Rebate Disablement

Category	Severity	Location	Status
Design Issue	Minor	referral_storage_contract/src/contracts/referral/ReferralStorage.sol (init-0 812): 156~165	Resolved

Description

The `setReferrerTier` function is responsible for assigning a rebate tier to a given referrer so downstream contracts can derive `totalRebate` and `discountShare` from that tier. However, the function does not verify that `_tierId` has been initialized through `setTier` before the assignment is made. As a result, an admin can bind a referrer to a non-configured tier, and later consumers such as `RebateDistributor` will read zero-valued rebate parameters for that referrer, causing rebate distribution to silently stop instead of failing explicitly.

ReferralStorage.sol

```
156     function setReferrerTier(address _referrer, uint256 _tierId)
157         external
158         override
159         onlyRole(ADMIN_ROLE)
160     {
161         if (_referrer == address(0)) revert ZeroAddress();
162
163         _referrerTiers[_referrer] = _tierId;
164         emit ReferrerTierSet(_referrer, _tierId);
```

Recommendation

Validate that the target tier has been initialized before assigning it, for example by tracking tier existence explicitly and reverting when `_tierId` refers to an undefined configuration.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding `_tierInitialized[_tierId]` check in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-09 | Missing Duplicate Recipient Validation Leads To Repeated Rebate Distribution

Category	Severity	Location	Status
Coding Issue	Minor	rebate_contract/src/contracts/core/referral/RebateDistributor.sol (init-0812): 237~249	Resolved

Description

The `batchSyncRebates` function is intended to synchronize rebate payouts in bulk by iterating over `users`, increasing `claimedRebates`, and transferring the corresponding `amounts[i]` to each recipient. However, the function does not detect duplicate addresses within the same batch, so the same user can appear multiple times and receive multiple transfers in one execution. This means an operational mistake or malformed batch input can directly result in overpayment and inflated `claimedRebates` records for that address. Similarly, `userCount` will also be overcounted.

```
237     for (uint256 i = 0; i < users.length; i++) {
238         if (amounts[i] > 0 && users[i] != address(0)) {
239             // Update claimed amount
240             claimedRebates[users[i]] += amounts[i];
241
242             // Transfer USDC to user
243             usdc.safeTransfer(users[i], amounts[i]);
244
245             totalAmount += amounts[i];
246             userCount++;
247         }
248     }
```

Recommendation

Reject batches containing duplicate recipient addresses, or aggregate repeated entries off-chain before submission so each address is processed at most once per batch.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding a check in the `for` loop to ensure that user addresses are in ascending order in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-10 | Missing Zero Address Validation

Category	Severity	Location	Status
Code Optimization	Minor	plp_contract/src/LiquidityVault.sol (init-0812): 103~105; rebate_contract/src/contracts/core/referral/RebateDistributor.sol (init-0812): 268~270; vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 843~845, 871~874	Resolved

Description

The address-setting functions in these contracts lack zero-address validation. Accidentally setting these addresses to the zero address may impair the availability and functionality of the contracts.

Recommendation

Add zero-address validation when setting these addresses.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding zero-address validation in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-11 | Incomplete `initialize()` Forces Mandatory Post-Deploy `reinitializer` Calls For Configuration

Category	Severity	Location	Status
Logical Issue	● Minor	vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 180~215, 871~874	● Resolved

Description

`initialize()` only configures the original Vault fields (USDC, signer, referral storage, EIP-712 domain, min deposit/withdraw). It does **not** set several later-added state variables that the current implementation depends on.

`Vault.sol`

```

180     function initialize(
181         address _usdc,
182         address _backendSigner,
183         address _referralStorage,
184         string memory _domainName,
185         string memory _domainVersion,
186         address _owner
187     ) external initializer {
188         __ReentrancyGuard_init();
189         __Pausable_init();
190         __Ownable_init(_owner);
191         __UUPSUpgradeable_init();
192
193         if (_usdc == address(0)) revert ZeroAddress();
194         if (_backendSigner == address(0)) revert ZeroAddress();
195         if (_owner == address(0)) revert ZeroAddress();
196         if (bytes(_domainName).length == 0) revert EmptyDomainName();
197         if (bytes(_domainVersion).length == 0) revert EmptyDomainVersion();
198
199         usdc = IERC20Metadata(_usdc);
200         backendSigner = _backendSigner;
201         NAME = _domainName;
202         VERSION = _domainVersion;
203
204         if (_referralStorage != address(0)) {
205             referralStorage = IReferralStorage(_referralStorage);
206         }
207
208         // Set default minimums (1 token unit in the token's decimals)
209         uint8 decimals = usdc.decimals();
210         minDeposit = 10 ** decimals;
211         minWithdraw = 10 ** decimals;
212
213         // Compute EIP-712 domain separator
214         DOMAIN_SEPARATOR = SignatureVerifier.computeDomainSeparator(NAME,
215             VERSION, block.chainid, address(this));
216     }

```

Variable	Default after initialize()	How it can be set
dailySettlementCreditCap	0	reinitializer(3) or setDailySettlementCreditCap()
dailyUserSettlementCreditCap	0	reinitializer(3) or setDailyUserSettlementCreditCap()
protocolFeeRecipient	address(0)	reinitializer(4) or setProtocolFeeRecipient()

Variable	Default after <code>initialize()</code>	How it can be set
		<code>ient()</code>
<code>liquidationManager</code>	<code>address(0)</code>	<code>setLiquidationManager()</code>
<code>plpVaultAddress</code>	<code>address(0)</code>	<code>reinitializeAddPLPAddress()</code> only (<code>reinitializer(5)</code>)

While settlement caps and the protocol-fee recipient can still be configured later via owner setters, `plpVaultAddress` has **no ordinary setter**. It can only be written through `reinitializeAddPLPAddress()`:

`Vault.sol`

```

871     function reinitializeAddPLPAddress(address _plpVault) external
reinitializer(5) onlyOwner {
872         plpVaultAddress = _plpVault;
873         emit PLPAddressSet(_plpVault);
874     }

```

Until that runs, `debitFromUser()` / `creditToUser()` always revert with `OnlyPLP()`, so **LiquidityVault cannot integrate with Vault after deployment**.

Impact

1. **PLP is non-functional immediately after proxy deployment** unless `reinitializeAddPLPAddress()` is executed in the same deployment batch.
2. **`reinitializer(5)` is effectively part of initialization**, not a true “later upgrade-only” hook. On a fresh deploy, the team must consume initialization version 5 just to finish setup.
3. With caps left at 0, positive settlement paths are blocked until owner/reinitializer configuration completes.

Recommendation

- extend `initialize()` script to accept `plpVaultAddress` (and optionally caps / fee recipient), or
- add an owner-gated `setPlpVaultAddress()` with appropriate one-shot semantics.

Alleviation

[Primit 2, 09/03/2026]:

The project team resolved this issue by initializing the corresponding state variables and addresses in `initialize()` in commit [c8253041c00a4f38881d55c0d121bad5842a8026](https://github.com/c8253041c00a4f38881d55c0d121bad5842a8026)

Although the current implementation does not present an issue, we would still like to remind the project team that initializing `liquidationManager` in `initialize()` results in a non-standard deployment sequence for deployments (as opposed to

upgrades), because `liquidationManager` requires the vault address to be set in its constructor.

If the `vault` and `liquidationManager` need to be redeployed, the `vault` proxy must first be deployed. The `vault` proxy address should then be provided as the constructor argument when deploying `liquidationManager`. Finally, the vault's `initialize()` function should be called with the deployed `liquidationManager` address as an input.

PRI-12 | `minWithdraw` Prevents Users From Fully Draining Their Vault Balance

Category	Severity	Location	Status
Logical Issue	● Minor	<code>vault_contract/src/contracts/core/vault/Vault.sol</code> (init-0812): 296~298	● Resolved

Description

`withdraw()` requires every withdrawal to satisfy `amount >= minWithdraw`:

`Vault.sol`

```
296         if (amount < minWithdraw) {
297             revert AmountBelowMinimum(amount, minWithdraw);
298         }
```

At initialization, `minWithdraw = 10 ** decimals` (1 USDC for native USDC). There is no "withdraw full balance" exception and no path allowing a final withdrawal below `minWithdraw`.

Impact

- If $0 < \text{balance} < \text{minWithdraw}$, the user **cannot withdraw at all**.
- Funds are not seized by the protocol, but recovery depends on the owner lowering `minWithdraw` or the user receiving additional credits and withdrawing in a larger combined amount.

Recommendation

We recommended adding a dedicated `withdrawAll()` path to clear dust.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding `withdrawAll()` function in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-13 | `permitCloseOperator()` Does Not Validate That `expiresAt` Is In The Future

Category	Severity	Location	Status
Logical Issue	Minor	<code>vault_contract/src/contracts/core/vault/Vault.sol</code> (init-0812): 665~689	Resolved

Description

`permitCloseOperator()` validates the relay `deadline` but never checks that `expiresAt` is greater than `block.timestamp`. A permit can therefore be recorded with an already-expired (or zero) operator approval window. The call still consumes `closeOperatorNonces[user]` and stores the value, but any subsequent `recordPositionCloseFor()` / `recordPositionCloseAndSettleFor()` attempt will immediately revert with `CloseOperatorApprovalExpired`.

`Vault.sol`

```
665     function permitCloseOperator(  
666         address user,  
667         address operator,  
668         uint64 expiresAt,  
669         uint256 deadline,  
670         bytes calldata signature  
671     ) external override {  
672         if (user == address(0)) revert ZeroAddress();  
673         if (operator == address(0)) revert ZeroAddress();  
674         if (block.timestamp > deadline) {  
675             revert SignatureExpired(deadline, block.timestamp);  
676         }  
677         // ...  
678         closeOperatorApprovalExpiries[user][operator] = expiresAt;
```

Recommendation

We recommended adding an `expiresAt` check before writing `closeOperatorApprovalExpiries`.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding an `expiresAt` check in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-14 Issue When `usedPositionBalanceSettlements` Is Already Consumed In `recordPositionCloseAndSettleFor`

Category	Severity	Location	Status
Logical Issue	Minor	<code>vault_contract/src/contracts/core/vault/Vault.sol</code> (init-0812): 808~817	Resolved

Description

In `_recordPositionCloseAndSettleFor`, when `usedPositionBalanceSettlements[params.closeId]` is already `true`, the function skips `_settlePositionBalance` (no balance change, no `withdrawNonces` increment) and `_transferProtocolFees` but still emits `PositionBalanceSettled`:

`Vault.sol`

```

808         if (usedPositionBalanceSettlements[params.closeId]) {
809             emit PositionBalanceSettled(
810                 params.closeId, params.user, 0, 0, 0, _balances[params.user],
            withdrawNonces[params.user]
811             );
812             return (0, 0);
813         }
814
815         (creditedAmount, debitedAmount) = _settlePositionBalance(params.user,
            params.balanceDelta, params.closeId);
816         _transferProtocolFees(params.closeId, params.user, params.tradingFee,
            params.fundingFee, params.borrowingFee);
817         return (creditedAmount, debitedAmount);

```

The normal settlement path only emits this event after applying the delta and bumping `withdrawNonces`. Here the event carries zero amounts and an unchanged nonce, which can mislead indexers into treating it as a new settlement and nonce invalidation when none occurred.

In addition, when `usedPositionBalanceSettlements` has already been consumed, `_transferProtocolFees` is skipped. We would like to confirm whether not charging protocol fees in this case is part of the intended design.

Recommendation

We recommended removing the `PositionBalanceSettled` emit in this branch and returning `(0, 0)` after the close audit events only. We would like to confirm whether not charging protocol fees in this case is part of the intended design.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by removing the `PositionBalanceSettled` emit in this branch in commit [7c98edf932bbbbb37c7af385f31dd892df67da19d](#).

In addition, regarding the issue that “when `usedPositionBalanceSettlements` has already been consumed, `_transferProtocolFees` is skipped,” the team confirmed that this is the intended design.

PRI-15 | Undocumented Calculation Precision In `_remainingCollateral`

Category	Severity	Location	Status
Inconsistency	● Minor	vault_contract/src/contracts/core/liquidation/LiquidationManager.sol (init -0812): 255~262	● Resolved

Description

In the `PositionSnapshot` struct, the precision of `sizeUsd`, `sizeTokens`, `collateral`, `accumulatedFunding`, and `accumulatedBorrowing` is not specified. Similarly, the precision of `price` in `OraclePrice` is also undocumented. Therefore, `_remainingCollateral()` needs to ensure that:

1. The calculated `positionValue` and `snap.sizeUsd` use the same precision.
2. `snap.collateral`, `pnl`, `snap.accumulatedFunding`, and `snap.accumulatedBorrowing` use the same precision.
3. The outer functions calling `_remainingCollateral()` use the correct precision when performing calculations with the returned data.

`LiquidationManager.sol`

```

255     function _remainingCollateral(PositionSnapshot memory snap, uint256
markPrice) private pure returns (int256) {
256         uint256 positionValue = (snap.sizeTokens * markPrice) / 1e18;
257         int256 pnl = snap.side == PositionSide.Long
258             ? int256(positionValue) - int256(snap.sizeUsd)
259             : int256(snap.sizeUsd) - int256(positionValue);
260
261         return int256(snap.collateral) + pnl - snap.accumulatedFunding - snap.
accumulatedBorrowing;
262     }

```

Recommendation

We recommend explicitly defining the precision of these parameters and documenting it in the documentation or code comments to prevent misuse.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding the corresponding precise conventions to the NatSpec in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](https://github.com/certik/vault-contract/commit/7c98edf932bbbb37c7af385f31dd892df67da19d).

PRI-16 | Stale `PositionSnapshot` Has No Expiry Before Liquidation

Category	Severity	Location	Status
Logical Issue	Minor	vault_contract/src/contracts/core/liquidation/LiquidationManager.sol (init-0812): 147~164	Resolved

Description

`updateSnapshot` does not enforce a maximum age on `snap.updatedAt`. An `Open` snapshot can remain on-chain indefinitely and can only be closed manually via `closeSnapshot`; users or bots holding a `backendSignature` may be unable to submit the close promptly for various reasons.

`LiquidationManager.sol`

```

147     function updateSnapshot(bytes32 positionId, PositionSnapshot calldata snap,
148         bytes calldata backendSignature)
149         external
150     {
151         ...
152         _snapshots[positionId] = snap;

```

`liquidate` only checks oracle price freshness, not snapshot freshness:

`LiquidationManager.sol`

```

188         if (block.timestamp > uint256(oraclePrice.updatedAt) + uint256(
189             oraclePrice.maxStaleness)) {
190             revert StaleOraclePrice(oraclePrice.updatedAt, oraclePrice.
191                 maxStaleness, block.timestamp);
192         }
193         int256 remaining = _remainingCollateral(snap, oraclePrice.price);

```

In this situation, a keeper can liquidate using a stale snapshot with a fresh mark price, leading to incorrect liquidation.

Recommendation

We recommended adding a maximum snapshot age check in `liquidate`, e.g. reverting when `block.timestamp > snap.updatedAt + maxSnapshotStaleness`, consistent with the existing oracle staleness check.

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding `maxSnapshotAge` check in the `liquidate()` in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-17 | `grantHandlerRole` / `revokeHandlerRole` Incompatible With `ADMIN_ROLE` -Only Callers

Category	Severity	Location	Status
Inconsistency	Minor	referral_storage_contract/src/contracts/referral/ReferralStorage.sol (init-0812): 85~86, 187~198	Resolved

Description

`grantHandlerRole` and `revokeHandlerRole` require `ADMIN_ROLE`, but internally call OpenZeppelin `grantRole` / `revokeRole`, which enforce the admin role of `HANDLER_ROLE`. The contract never calls `_setRoleAdmin(HANDLER_ROLE, ADMIN_ROLE)`, so `HANDLER_ROLE` remains administered by `DEFAULT_ADMIN_ROLE`:

ReferralStorage.sol

```
85         _grantRole(DEFAULT_ADMIN_ROLE, _admin);
86         _grantRole(ADMIN_ROLE, _admin);
```

ReferralStorage.sol

```
187     function grantHandlerRole(address _handler) external onlyRole(ADMIN_ROLE) {
188         if (_handler == address(0)) revert ZeroAddress();
189         grantRole(HANDLER_ROLE, _handler);
190     }
191
192     /**
193      * @notice Revoke handler role from a contract
194      * @param _handler The address of the handler contract
195      */
196     function revokeHandlerRole(address _handler) external onlyRole(ADMIN_ROLE)
197     {
198         revokeRole(HANDLER_ROLE, _handler);
199     }
```

An account with `ADMIN_ROLE` but without `DEFAULT_ADMIN_ROLE` passes the outer check yet reverts inside `grantRole` / `revokeRole`. The wrapper therefore does not match the intended delegated-admin design; handler rotation via these functions requires `DEFAULT_ADMIN_ROLE` (or a direct `grantRole` / `revokeRole` call).

Recommendation

We recommended calling `_setRoleAdmin(HANDLER_ROLE, ADMIN_ROLE)` during initialization, or replacing `grantRole` / `revokeRole` with `_grantRole` / `_revokeRole` inside the wrapper functions.

■ Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by replacing `grantRole` / `revokeRole` with `_grantRole` / `_revokeRole` inside the wrapper functions in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-23 | `settlePositionBalance()` Unexpectedly Invalidates Pending Withdraw Signatures

Category	Severity	Location	Status
Logical Issue	● Minor	<code>vault_contract/src/contracts/core/vault/Vault.sol</code> (init-0812): 513	● Acknowledged

Description

Withdrawals use the shared per-user counter `withdrawNonces` in the EIP-712 `Withdraw(...)` signature.

`_settlePositionBalance()` also increments `withdrawNonces[user]` even though it is a position PnL settlement path, not a withdrawal. Any unused backend-signed withdraw message for that user therefore becomes invalid once `settlePositionBalance()` or `recordPositionCloseAndSettleFor()` (which calls the same internal function) executes.

Unlike `settleLiquidation()`, this path has **no documentation** stating that nonce invalidation is intended. Integrators and backends are likely to assume `withdrawNonces` only advances on `withdraw()`.

`Vault.sol`

```
513         withdrawNonces[user] += 1;
```

This will cause a previously unused withdrawal signature held by a user to be silently invalidated after position balance settlement or a close-and-settle operation.

Recommendation

We would like to confirm with the protocol team whether this is the intended design. If so, this behavior should be clearly disclosed to users in the documentation or code comments.

Alleviation

[Primit 2, 08/28/2026]: Status: Acknowledged (Discussion).

Confirming intent and satisfying Certik's disclosure request. (1) The nonce-invalidation side effect is intentional: `settlePositionBalance` (and by extension `recordPositionCloseAndSettleFor` via the internal `_settlePositionBalance` path) and `settleLiquidation` all bump `withdrawNonces[user]` because a settlement changes the user's spendable `Vault._balances`. If we did not invalidate, a previously-issued backend-signed withdraw for an amount that was valid pre-settlement could be spent against a post-settlement balance that is now smaller, forcing a revert deep inside withdraw rather than a clean re-sign at the backend. Bumping the nonce is the on-chain signal to the off-chain backend that the cached withdraw signature is stale and a new one must be issued against the current balance. This is the same design as `settleLiquidation`, where Certik acknowledged the intent; PRI-23 correctly points out that `settlePositionBalance` and `recordPositionCloseAndSettleFor` share this behaviour without having said so in the code. (2) Satisfying the disclosure request: PR <https://github.com/Primit-io/primit-avax-contracts/pull/24> adds dedicated natspec blocks on `settleLiquidation` and `settlePositionBalance` stating the design contract explicitly. The `settleLiquidation` natspec now says "INTENTIONALLY invalidates the user's pending withdraw

signatures by incrementing `withdrawNonces[user]` with the rationale and a note that the invalidated nonce is echoed in the `LiquidationSettled` event for off-chain indexers. The `settlePositionBalance` natspec now says the same thing and explicitly calls out that `recordPositionCloseAndSettleFor` inherits this via `_settlePositionBalance`, with the operational directive "Off-chain backends MUST NOT cache withdraw signatures across settlement events." The invalidated nonce is likewise echoed in the `PositionBalanceSettled` event's `invalidatedNonce` field so any indexer or backend session cache can pick it up in the same block as the settlement. This is a documentation-only change with no behaviour change; the full `vault_contract` Foundry suite (95 of 95 tests) continues to pass. Deployment: intentionally not broadcast to mainnet as a standalone Discussion-severity docs-fix. The natspec is on the `v2/certik-preliminary-fixes` branch and will be picked up by the next scheduled Vault release (next CertiK batch that ships a substantive fix, or the SEC-2026-0824-001 backend / contract ABI reconciliation, whichever comes first).

PRI-26 | Confirmation On Fee And Maintenance Margin

Category	Severity	Location	Status
Logical Issue	● Minor	vault_contract/src/contracts/core/liquidation/LiquidationManager.sol (init-0812): 192~200; vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 913~956	● Resolved

Description

In `LiquidationManager`, the `_snapshots` used by `liquidate()` are not subject to proper rate-bound checks when they are provided through `updateSnapshot()`. For example, there is no upper-bound check on `liquidationFeeRate` (e.g., ensuring that it does not exceed 100%), nor is there a lower-bound check on `maintenanceMarginRate`; only zero-value checks are performed. This may result in excessively high fees or insufficient maintenance margin requirements. We would like to confirm with the team whether these validations are expected to be performed by upstream components and whether this is the intended design.

LiquidationManager.sol

```

192         int256 remaining = _remainingCollateral(snap, oraclePrice.price);
193         uint256 maintenanceMargin = (snap.sizeUsd * snap.maintenanceMarginRate)
    / 1e18;
194         if (remaining >= int256(maintenanceMargin)) {
195             revert NotLiquidatable(remaining, int256(maintenanceMargin));
196         }
197
198         uint256 requestedDebit = (snap.sizeUsd * snap.liquidationFeeRate) /
1e18;
199         bool clearBalance = remaining < 0;
200         liquidationKey = keccak256(abi.encodePacked(positionId, snap.nonce,
    oraclePrice.price, oraclePrice.updatedAt));

```

In `Vault`, `recordPositionCloseAndSettleFor()` does not perform numerical validation on `params.tradingFee`, `params.fundingFee`, or `params.borrowingFee`. As a result, the fee calculated in `_transferProtocolFees` could potentially be excessively high. We would like to confirm with the team whether these validations rely on upstream checks and whether this is the intended design.

Vault.sol

```
913     function _transferProtocolFees(
914         bytes32 closeId,
915         address user,
916         uint256 tradingFee,
917         int256 fundingFee,
918         uint256 borrowingFee
919     ) internal {
920         uint256 feeAmount = _computeProtocolFeeAmount(tradingFee, fundingFee,
borrowingFee);
921         if (feeAmount == 0) {
922             emit ProtocolFeeAccrued(closeId, user, address(usdc), tradingFee,
fundingFee, borrowingFee, 0);
923             return;
924         }
925
926         address token = address(usdc);
927         address recipient = protocolFeeRecipient;
928         if (recipient == address(0)) revert ProtocolFeeRecipientNotSet();
929
930         usdc.safeTransfer(recipient, feeAmount);
931         emit ProtocolFeeAccrued(closeId, user, token, tradingFee, fundingFee,
borrowingFee, feeAmount);
932     }
933
934     function _computeProtocolFeeAmount(uint256 tradingFee, int256 fundingFee,
uint256 borrowingFee)
935         internal
936         pure
937         returns (uint256)
938     {
939         if (borrowingFee > type(uint256).max - tradingFee) revert
InvalidFeeAmounts();
940         uint256 positiveFees = tradingFee + borrowingFee;
941         uint256 maxInt = uint256(type(int256).max);
942         if (positiveFees > maxInt) revert InvalidFeeAmounts();
943
944         if (fundingFee >= 0) {
945             uint256 positiveFunding = uint256(fundingFee);
946             if (positiveFees > maxInt - positiveFunding) revert
InvalidFeeAmounts();
947             return positiveFees + positiveFunding;
948         }
949
950         if (fundingFee == type(int256).min) revert InvalidFeeAmounts();
951         uint256 fundingCredit = uint256(-fundingFee);
952         if (fundingCredit >= positiveFees) {
953             return 0;
954         }
955         return positiveFees - fundingCredit;
956     }
```

Recommendation

We would like to confirm with the team whether these validations are expected to be performed by upstream components and whether this is the intended design.

Alleviation

[Primit 2, 08/24/2026]: Status: Acknowledged.

We agree with Certik's observation and have added on-chain defense-in-depth bounds on both sides so the contract no longer relies solely on upstream backend validation for rate / fee sanity. Fix PR <https://github.com/Primit-io/primit-avax-contracts/pull/27> lands the code on the v2/certik-preliminary-fixes branch. LiquidationManager side: three new owner-configurable bounds -- maxLiquidationFeeRate (default 5e16 = 5%), minMaintenanceMarginRate (default 1e15 = 0.1%), maxMaintenanceMarginRate (default 5e17 = 50%) -- are checked inside _validateSnapshot before a backend-signed snapshot is accepted through updateSnapshot. Errors LiquidationFeeRateOutOfBounds and MaintenanceMarginRateOutOfBounds carry the provided value and the active bound so keepers and off-chain monitors can diagnose rejections precisely. Two owner-only setters (setMaxLiquidationFeeRate and setMaintenanceMarginRateBounds) allow the bounds to be returned as the fee schedule evolves; both setters reject zero or inverted values so the protection cannot be disabled through the setter. Vault side: new maxProtocolFeeAmount storage (USDC decimals, aggregate cap per settlement) is checked inside _transferProtocolFees after _computeProtocolFeeAmount. To keep the already-live upgradeable proxy backward-compatible without a reinitializer bump, the field starts at 0 (cap disabled) and the owner explicitly enables it by calling setMaxProtocolFeeAmount with a positive value in a follow-up transaction after any future Vault upgrade; the setter rejects zero so once enabled the cap cannot be disabled through the setter. Errors ProtocolFeeExceedsMax and ZeroMaxProtocolFeeAmount carry the diagnostic data. Storage layout: __gap reduced from 31 to 30 slots to account for maxProtocolFeeAmount. TDD coverage: 13 new tests (9 on the LM side, 4 on the Vault side) assert defaults, per-bound reject paths, setter emit + update, and setter rejection of invalid inputs; also assert that a tightened bound retroactively gates subsequent updateSnapshot calls, that the existing snap parameters used by every other test are still within the default bounds (regression), and that non-owner callers cannot touch any of the setters. Full vault_contract Foundry suite: 108 of 108 tests pass (was 95 of 95). Deployment: intentionally not broadcast to mainnet as a standalone Discussion-severity fix. Consistent with our deferral for PRI-16 / PRI-18 / PRI-21 / PRI-23 / PRI-24 / PRI-25: the mainnet LiquidationManager path is not functional end-to-end pending the SEC-2026-0824-001 backend / contract ABI reconciliation, and shipping this defense-in-depth alone would put it into code paths no real user or backend can reach today. Both fixes will be picked up by the next canonical release, at which point we will update this reply to Resolved with the deployed implementation address and upgrade transaction hash.

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by adding the corresponding upper and lower bound checks for `maintenanceMarginRate`, an upper bound check for `liquidationFeeRate`, and an upper bound check for `feeAmount` in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](https://github.com/Primit-io/primit-avax-contracts/commit/7c98edf932bbbb37c7af385f31dd892df67da19d)

PRI-18 | Redundant Code Components

Category	Severity	Location	Status
Volatile Code	● Informational	vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 561, 570~571	● Resolved

Description

The linked statements do not affect the functionality of the codebase and appear to be either leftovers from test code or older functionality.

Recommendation

It is recommended to remove the redundant statements for the production environment.

Alleviation

[Primit 2, 08/28/2026]: Status: Acknowledged.

We agree with the finding. Code cleanup landed in PR <https://github.com/Primit-io/primit-avax-contracts/pull/21> on the v2/certik-preliminary-fixes branch. The three redundant statements identified by CertiK are removed: the private constant `_VAULT_VERSION = 0x56415556` (a "VAUV" ASCII deployment marker that was never read anywhere on the contract surface), the local `uint256 _v = _VAULT_VERSION` at the top of `_setReferralCode`, and the inline assembly `write assembly { _v := add(_v, number()) }`. `_v` was never read after the assembly write, so the sequence was a zero-value marker leftover with no runtime effect. Full vault_contract Foundry suite: 95 of 95 tests pass, confirming no behaviour change. Status is Acknowledged rather than Resolved because we are intentionally deferring the mainnet redeploy for this Informational-severity cleanup: shipping it as a standalone Vault UUPS upgrade would consume admin-key operational overhead and gas for zero security benefit, so the cleaned code is queued for the next scheduled Vault release, which will either be the next CertiK batch that ships a substantive fix or the SEC-2026-0824-001 canonical LiquidationManager / Vault reconciliation, whichever comes first. At that point we will update this reply to Resolved with the new deployed implementation address and upgrade transaction hash.

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by removing redundant code components in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#).

PRI-19 | Stale Role Entries Remain In `_roles` After Last Member Is Revoked

Category	Severity	Location	Status
Design Issue	● Informational	shared/role/RoleStore.sol (init-0812): 116~121	● Resolved

Description

When a role is granted, `_grantRole` adds the role identifier to the global `_roles` set. When the last member is revoked, `_revokeRole` removes the account from `_roleMembers` and `_accountRoles`, but does not remove the role from `_roles` if it no longer has any members.

`RoleStore.sol`

```
116     function _revokeRole(bytes32 role, address account) internal {
117         if (_roleMembers[role].remove(account)) {
118             _accountRoles[account].remove(role);
119             emit RoleRevoked(role, account, msg.sender);
120         }
121     }
```

As a result, `_roles` can accumulate entries for roles with zero members.

Impact

- `_roles` grows monotonically and never shrinks, causing minor storage bloat over time.
- Any future logic that iterates `_roles` may treat empty roles as active.

Recommendation

After revoking a member, remove the role from `_roles` when `_roleMembers[role].length() == 0`:

Alleviation

[Primit 2, 08/28/2026]: The team heeded the advice and resolved the issue by removing the role from `_roles` when `_roleMembers[role].length() == 0` in commit [7c98edf932bbbb37c7af385f31dd892df67da19d](#)

PRI-20 | Concerns Regarding The Vault'S On-Chain State

Category	Severity	Location	Status
Operational Risk	● Informational	rebate_contract/src/contracts/core/referral/RebateDistributor.sol (init-0812): 149~161; vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 221~226, 871~874	● Resolved

Description

In the on-chain Vault contract `0xdf19d902fcb6295366a06620f76dcb49e686c403`, `reinitializeAddPLPAddress(address _plpVault)` was called immediately after deployment in [transaction 0x645ddf5eea606fdc0a0b4b6f5767f0dfe3bf3f2bb00150fc4eefa86e119a774d](#). This directly skips the initialization steps performed by `reinitializeEip712DomainVersion()`, `reinitializeSettlementCaps()`, and `reinitializeProtocolFees()`, and these functions can no longer be called afterward.

Among them, the inability to call `reinitializeEip712DomainVersion()` means that `VERSION` and `DOMAIN_SEPARATOR` can no longer be reconfigured, while the other parameters can still be updated through their corresponding setter functions. We would like to confirm with the protocol team whether this is the intended behavior.

For the on-chain RebateDistributor contract `0x726c54c38119d66e8d2b5d7339b57d8b47e48a3b`, the authority to call `reinitializeEip712DomainVersion()` is still retained. The documentation states:

v1 EIP-712 domain versions are pinned to "1.0.0" for Vault and Rebate, and "1" for PLP. Any future upgrade must not change these (off-chain signatures are backward-compatible).

Therefore, we would like to remind the protocol team to ensure that the `EIP-712 domain versions` remain unchanged when calling `reinitializeEip712DomainVersion()`.

Recommendation

We would like to confirm with the protocol team whether this is the intended behavior.

Alleviation

[Primit 2, 08/24/2026]: Status: Acknowledged.

We confirm that the observed on-chain state on the Vault at `0xdf19d902fcb6295366a06620f76dcb49e686c403` is intentional and matches our design. When the D6 upgrade batch (2026-07-16) migrated the Vault to the PLP-integrated implementation, we called `reinitializeAddPLPAddress (reinitializer(5))` directly to wire the PLP LiquidityVault, which correctly moved `_initialized` from 1 to 5 and, as CertiK notes, made `reinitializeEip712DomainVersion / reinitializeSettlementCaps / reinitializeProtocolFees` non-callable going forward. For the two settings that are still practically mutable, coverage is preserved via the corresponding admin setters `setDailySettlementCreditCap` and `setDailyUserSettlementCreditCap` for the settlement caps, and `setProtocolFeeRecipient` for the fee recipient. so no operational capability is lost. The one setting that is now permanently locked is `Vault.VERSION` and its derived `DOMAIN_SEPARATOR`: this is not only acceptable but is exactly

the outcome we want. Our EIP-712 domain versions are pinned to "1.0.0" for Vault and Rebate and "1" for PLP as CertiK correctly quoted from our documentation, because off-chain signatures issued to users are backward-compatibility-critical and any change to VERSION would silently invalidate every outstanding withdraw / close signature. Having the Vault's VERSION locked at the reinitializer level is therefore an effective, defense-in-depth guarantee that the "must not change" contract is enforced by on-chain semantics and not solely by admin discipline. Regarding the RebateDistributor at 0x726c54c38119d66e8d2b5d7339b57d8b47e48a3b: yes, reinitializeEip712DomainVersion is still callable there because the RD proxy did not consume its reinitializer past v2. We confirm the operational commitment that the RD domain version will remain "1.0.0" in perpetuity, matching the Vault contract. This commitment is enforced by admin discipline today and will be further hardened once the PRI-01 / PRI-02 governance migration completes, at which point the RD owner will be a Gnosis Safe 3-of-5 multisig behind a 48-hour TimelockController; a hypothetical domain-version change would then require Safe quorum and 48-hour on-chain notice, which is enough surface for both the community and CertiK to react. If preferred, we can add a natspec warning on RebateDistributor.reinitializeEip712DomainVersion in the next release to make the "keep at 1.0.0" contract explicit in-source, but we do not think a code-level revert on version change is warranted given the Safe+Timelock backstop. No mainnet upgrade is required for this finding.

PRI-22 | Confirmation On Incomplete LiquidityVault

Category	Severity	Location	Status
Design Issue	● Informational	plp_contract/src/LiquidityVault.sol (init-0812): 20~26	● Acknowledged

Description

Currently, after depositing into `LiquidityVault` via `deposit()`, users can only withdraw their funds at a 1:1 ratio through `withdraw()` upon maturity, with no apparent source of yield and yield accounting for users. In addition, `applyFeeShare` and `applyMarketPnL` only modify `totalAssets` and, unlike `deposit` and `withdraw`, do not involve any corresponding token inflows or outflows. Meanwhile, the declared `PERFORMANCE_FEE_BP` is unused, and no profits are transferred to the `BufferPool`.

In the documentation, the team states that the lack of a yield source is expected behavior.

`injectYield(uint256)` and `absorbLoss(uint256)` are defined at protocol design level but not implemented in v1. PLP v1 is share-only (no NAV accounting). `plpShares` = deposited amount (1:1). `totalAssets` and yield accounting are reserved for v1.1.

However, we would like to confirm with the team whether the absence of token transfers in `applyFeeShare` and `applyMarketPnL`, as well as the lack of profit transfers to the `BufferPool`, is also part of the intended design.

In addition, `deposit` and `withdraw` call `debitFromUser` and `creditToUser` on the Vault, respectively, thereby affecting the user's balance in the Vault. In the Vault, aside from `withdraw`, other operations that modify a user's balance invalidate the existing `withdrawNonces`. However, `debitFromUser` and `creditToUser` do not invalidate the nonce. We would like to confirm with the team whether this is the intended design.

Recommendation

We would like to confirm with the protocol team whether this is the intended design.

Alleviation

[Primit 2, 08/24/2026]: Status: Discussion.

Both design points CertiK raised are intentional; we confirm them below. (1) `applyFeeShare` and `applyMarketPnL` modify only `totalAssets` and do not move USDC, and `PERFORMANCE_FEE_BP` is declared but not wired into a transfer to `BufferPool`: this is the v1 design contract already stated in our documentation. PLP v1 is deliberately share-only with `plpShares[user][tier]` equal to the deposited USDC amount at a 1-to-1 ratio, and NAV / real yield accounting are reserved for v1.1. The absence of on-chain USDC transfers in the two `apply*` operator paths is a direct consequence of that: there is no yield source in v1, so there is nothing to transfer. `applyFeeShare` exists to track the notional fee-share credit that will accrue to the buffer in v1.1, and `applyMarketPnL` exists to track the notional market P&L against `totalAssets` for the same reason; both operate against the internal accounting number `totalAssets` rather than the token because the token movement is a v1.1 concern. `PERFORMANCE_FEE_BP` is set to 1000 (10 percent) so the constant is already correct once v1.1 wires the actual profit

routing, but under v1 there is no realised profit to route, so no transfer to BufferPool is emitted. This matches the documentation quote CertiK included: "injectYield and absorbLoss are defined at the protocol design level but not implemented in v1; PLP v1 is share-only (no NAV accounting); plpShares equals deposited amount 1-to-1; totalAssets and yield accounting are reserved for v1.1." (2) LiquidityVault.deposit and LiquidityVault.withdraw call Vault.debitFromUser and Vault.creditToUser respectively, and these two paths intentionally do NOT invalidate the user's withdrawNonces, whereas settlePositionBalance / recordPositionCloseAndSettleFor / settleLiquidation do. The semantic difference is what drives the design: settlePositionBalance and settleLiquidation are position-PnL settlement operations that can shrink the user's effective spendable balance in ways the backend must re-sign to account for (a previously issued withdraw signature for amount X may now be over the user's remaining balance, and rather than let the user try to spend the stale sig and revert deep inside withdraw, we bump withdrawNonces so the backend must re-issue). By contrast, LiquidityVault.deposit is a same-account balance transfer: the user's vault balance decreases by amount, but the amount does not disappear from the user's overall economic position, it moves into plpShares under the same user's address. Any previously issued withdraw signature for amount Y still spends validly as long as Y is less than or equal to the user's post-transfer Vault._balances; if Y exceeds the remaining balance, Vault.withdraw naturally reverts InsufficientBalance and the stale signature is effectively self-invalidated on chain without needing to burn a nonce slot. LiquidityVault.withdraw is symmetric: creditToUser increases the user's Vault balance, and a larger balance cannot cause a pending withdraw signature to consume more than the signature already authorised. Therefore no nonce bump is required on either PLP path, and adding one would only add gas cost without changing the security envelope. If it would help future auditors, we can add a natspec note on debitFromUser and creditToUser in the next contract release stating the design contract explicitly ("balance transfer, not a settlement; does not consume withdrawNonces because the pending-signature amount is fixed and any over-spend is caught by InsufficientBalance"), but no code change is needed to satisfy this finding.

PRI-24 Confirmation On `params.closeId` In `recordPositionCloseAndSettleFor()`

Category	Severity	Location	Status
Design Issue	● Informational	vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 497, 791, 815	● Acknowledged

Description

In `recordPositionCloseAndSettleFor()`, `params.closeId` is used both as the key for the `usedCloseIds[]` replay-protection mapping and as the key for the `usedPositionBalanceSettlements[]` replay-protection mapping.

`Vault.sol`

```
791         usedCloseIds[params.closeId] = true;
```

`Vault.sol`

```
815         (creditedAmount, debitedAmount) = _settlePositionBalance(params.user,
params.balanceDelta, params.closeId);
```

`Vault.sol`

```
497         usedPositionBalanceSettlements[settlementKey] = true;
```

These two replay-protection mechanisms appear to represent different domains and therefore should seemingly use different keys. Accordingly, `recordPositionCloseAndSettleFor()` may need to accept two separate keys for validation. We would like to confirm with the team whether the current behavior is intentional.

Recommendation

We would like to confirm with the team whether the current behavior is intentional.

Alleviation

[Primit 2, 08/24/2026]: Status: Acknowledged (Discussion).

We confirm the current behaviour is intentional and have made the design contract explicit in-source. Using `params.closeId` as both the audit replay key (`usedCloseIds[closeId]`) and the settlement replay key (`usedPositionBalanceSettlements[closeId]`, accessed via `_settlePositionBalance`) is a deliberate simplification: a single canonical business identifier represents one position close, and locking both guards to the same key means the audit and settlement replay domains cannot drift out of sync. Two properties depend on this dual-key design and would break if we split them: (a) a completed record-and-settle cannot be split-replayed as either a bare audit re-emit or a bare balance re-settle,

because either half of a replay attempt hits an already-consumed guard; (b) the PRI-14 fallback path continues to work cleanly, where a backend has already consumed the settlement side via a direct `settlePositionBalance(user, delta, closeId)` call and an AA operator later submits `recordPositionCloseAndSettleFor` with the same `closeId`, and the flow correctly emits the audit events while skipping the double-charge on the settlement side without the AA caller having to know a second unrelated key. Splitting into two independent keys would give a very small domain-hygiene gain in exchange for a real operational complexity increase: every integrator and every off-chain component would have to carry two independent identifiers for the same close event, keep them in sync at signing time, and re-derive the correct pair when reconciling with the PRI-14 fallback branch. We judged the current single-key design to be the better trade-off. Fix PR <https://github.com/Primit-io/primit-avax-contracts/pull/25> extends the existing natspec on `recordPositionCloseAndSettleFor` with an explicit "dual-key design contract" block stating the rationale in-source (including the PRI-14 cross-reference), so future auditors and integrators can see the intent without having to reconstruct it from the call graph. This is a documentation-only change with no behaviour change; the full vault_contract Foundry suite (95 of 95 tests) continues to pass. Deployment: intentionally not broadcast to mainnet as a standalone Discussion-severity docs-fix. The natspec is on the v2/certik-preliminary-fixes branch and will be picked up by the next scheduled Vault release.

PRI-25 Confirmation On Access Control For `_setPositionBalance()` Calls

Category	Severity	Location	Status
Design Issue	● Informational	vault_contract/src/contracts/core/vault/Vault.sol (init-0812): 478~480, 780~789	● Acknowledged

Description

In `setPositionBalance()`, calling `_setPositionBalance()` requires the caller to be either the `liquidationManager` or the `backendSigner`.

Vault.sol

```
478         if (msg.sender != liquidationManager && msg.sender != backendSigner) {
479             revert UnauthorizedLiquidationManager(msg.sender);
480         }
```

In `recordPositionCloseAndSettleFor()`, however, the caller must be an authorized close operator and provide a signature from the `backendSigner`.

Vault.sol

```
780 @>         uint64 expiresAt = closeOperatorApprovalExpiries[params.user][msg.sender];
781 @>         if (expiresAt < block.timestamp) {
782             revert CloseOperatorApprovalExpired(params.user, msg.sender, expiresAt, block.timestamp);
783         }
784
785         if (usedCloseIds[params.closeId]) {
786             revert PositionCloseAlreadyRecorded(params.closeId);
787         }
788
789 @>         _verifyPositionCloseSettlement(params);
```

Although both functions ultimately perform the same underlying `_setPositionBalance()` operation, their access control mechanisms differ significantly. We would like to confirm with the protocol team whether this difference is part of the intended design.

Recommendation

We would like to confirm with the protocol team whether this difference is part of the intended design.

I Alleviation

[Primit 2, 08/24/2026]: Status: Acknowledged (Discussion).

We confirm the different access-control mechanisms on the two settlement paths are intentional and have made the design contract explicit in-source. The two paths serve two distinct UX / gas-payer patterns while intentionally converging on a single `_settlePositionBalance` internal for balance-mutation accounting: (1) `settlePositionBalance` is the BACKEND-DIRECT path, gated on `msg.sender == liquidationManager || msg.sender == backendSigner`. This is used when protocol infrastructure drives the transaction: the LiquidationManager after a verified oracle-priced liquidation, or the backend keeper posting a close-event pnl delta. No user permit is required because the caller is a system role, not a user-delegated operator; the backend or LM pays gas. (2) `recordPositionCloseAndSettleFor` is the AA-OPERATOR path, gated on `closeOperatorApprovalExpiries[user][msg.sender] >= block.timestamp` (with the PRI-21 self-submit shortcut when `msg.sender == user`) plus a backend signature over the full close-settlement payload verified inside `_verifyPositionCloseSettlement`. This is a Permit2-style flow: the user signs once via `permitCloseOperator` to authorize an operator (typically a 4337 bundler or a delegated close-relayer), the operator relays many close-and-settle transactions and pays gas, and the backend co-signature binds `balanceDelta` so the operator cannot pick a different credit/debit. Two access models exist because two callers with different trust profiles legitimately need to settle balances: the trusted system roles vs a user-delegated operator with a bounded permit window and a per-tx backend counter-signature. Cross-path replay is prevented by the shared `usedPositionBalanceSettlements` mapping keyed by `closeId` (per the PRI-24 dual-key design already documented), so no `closeId` can be settled twice regardless of which path was used first, and the PRI-14 fallback where a direct-path settle is followed by an AA record-and-settle for the same `closeId` still emits the audit events cleanly while skipping the double-charge. Fix PR <https://github.com/Primit-io/primit-avax-contracts/pull/26> adds an explicit "PRI-25 access model" natspec block on both entry points, cross-referencing each other and labelling the paths "BACKEND-DIRECT" and "AA-OPERATOR" respectively, so a future auditor or integrator can see the intent inline without reconstructing it from the modifier chain. This is a documentation-only change with no behaviour change; the full `vault_contract` Foundry suite (95 of 95 tests) continues to pass. Deployment: intentionally not broadcast to mainnet as a standalone Discussion-severity docs-fix. The natspec is on the `v2/certik-preliminary-fixes` branch and will be picked up by the next scheduled Vault release.

OPTIMIZATIONS | PRIMIT 2

ID	Title	Category	Severity	Status
PRI-21	<code>recordPositionCloseFor()</code> Requires Self- Authorization Via <code>permitCloseOperator()</code>	Code Optimization	Optimization	● Acknowledged

PRI-21 | recordPositionCloseFor() Requires Self-Authorization Via permitCloseOperator()

Category	Severity	Location	Status
Code Optimization	● Optimization	vault_contract/src/contracts/core/vault/Vault.sol (init-081 2): 722~725, 780~783	● Acknowledged

Description

`recordPositionCloseFor()` always requires a valid operator approval in `closeOperatorApprovalExpiries[user][msg.sender]`, even when `msg.sender == user`. A user who wants to submit their own close cannot call this function directly; they must first relay a separate `permitCloseOperator(user, user, ...)` transaction to authorize themselves. That adds an extra step, consumes `closeOperatorNonces`, and increases gas and UX friction.

`Vault.sol`

```

722         uint64 expiresAt = closeOperatorApprovalExpiries[user][msg.sender];
723         if (expiresAt < block.timestamp) {
724             revert CloseOperatorApprovalExpired(user, msg.sender, expiresAt,
block.timestamp);
725         }

```

The same pattern applies to `recordPositionCloseAndSettleFor()`.

Recommendation

We recommended treating `msg.sender == user` as an allowed caller without a prior `permitCloseOperator()` entry—for example by skipping the `closeOperatorApprovalExpiries` check when the submitter is the `user`.

Alleviation

[Primit 2, 08/24/2026]: Status: Acknowledged.

We agree with the recommendation and applied it verbatim. Code cleanup landed in PR <https://github.com/Primit-io/primit-avax-contracts/pull/23> on the v2/certik-preliminary-fixes branch. Both `recordPositionCloseFor` and `_recordPositionCloseAndSettleFor` now skip the `closeOperatorApprovalExpiries` check when `msg.sender == user`, so a position owner submitting their own close does not need to first relay a self-`permitCloseOperator` transaction. Third-party operators still require a live approval, so the delegated-close-operator path is preserved unchanged and no security guarantee is weakened. TDD coverage in `vault_contract/test/VaultCloseOperatorPermit.t.sol` asserts three behaviours: (a) `recordPositionCloseFor` called by the position owner directly succeeds without any prior permit (test explicitly asserts `closeOperatorApprovalExpiries[user][user] == 0` before the call), (b) `recordPositionCloseAndSettleFor` called by the position owner directly succeeds without any prior permit and correctly debits the user balance and updates the `usedCloseIds / usedPositionBalanceSettlements` guards, and (c) a third-party operator without a permit still reverts

CloseOperatorApprovalExpired (regression guard so the self-submit exemption cannot leak into third-party paths). Full vault_contract Foundry suite: 98 of 98 tests pass. Status is Acknowledged rather than Resolved because we are intentionally deferring the mainnet redeploy for this Optimization-severity fix, matching the deferral we have already communicated for PRI-16 and PRI-18: the mainnet close-and-settle path is not currently exercised end-to-end (see the SEC-2026-0824-001 internal architecture ticket referenced in the PRI-16 reply), so shipping this alone would put the fix into a code path no real user or backend can reach; the change will be picked up by the next canonical Vault release once the backend / contract ABI reconciliation lands. At that point we will update this reply to Resolved with the deployed implementation address and upgrade transaction hash.

APPENDIX | PRIMIT 2

Audit Scope

Primit-io/primit-contracts-v2-audit

 plp_contract/src/LiquidityVault.sol
 rebate_contract/src/contracts/core/referral/RebateDistributor.sol
 referral_storage_contract/src/contracts/referral/ReferralStorage.sol
 vault_contract/src/contracts/core/vault/Vault.sol
 shared/role/RoleStore.sol
 vault_contract/src/contracts/core/liquidation/LiquidationManager.sol
 plp_contract/src/BufferPool.sol
 rebate_contract/src/contracts/libraries/SignatureVerifier.sol
 rebate_contract/src/contracts/referral/IReferralStorage.sol
 referral_storage_contract/src/contracts/referral/IReferralStorage.sol
 shared/error/Errors.sol
 shared/price/Price.sol
 shared/role/Role.sol
 vault_contract/src/contracts/libraries/SignatureVerifier.sol
 vault_contract/src/contracts/referral/IReferralStorage.sol
 plp_contract/src/BufferPool.sol
 plp_contract/src/LiquidityVault.sol
 rebate_contract/src/contracts/core/referral/RebateDistributor.sol
 rebate_contract/src/contracts/libraries/SignatureVerifier.sol

Primit-io/primit-contracts-v2-audit

	rebate_contract/src/contracts/referral/IReferralStorage.sol
	referral_storage_contract/src/contracts/referral/IReferralStorage.sol
	referral_storage_contract/src/contracts/referral/ReferralStorage.sol
	shared/error/Errors.sol
	shared/price/Price.sol
	shared/role/Role.sol
	shared/role/RoleStore.sol
	vault_contract/src/contracts/core/liquidation/LiquidationManager.sol
	vault_contract/src/contracts/core/vault/Vault.sol
	vault_contract/src/contracts/libraries/SignatureVerifier.sol
	vault_contract/src/contracts/referral/IReferralStorage.sol
	plp_contract/src/BufferPool.sol
	plp_contract/src/LiquidityVault.sol
	rebate_contract/src/contracts/core/referral/RebateDistributor.sol
	rebate_contract/src/contracts/libraries/SignatureVerifier.sol
	rebate_contract/src/contracts/referral/IReferralStorage.sol
	referral_storage_contract/src/contracts/referral/ReferralStorage.sol
	referral_storage_contract/src/contracts/referral/IReferralStorage.sol
	shared/error/Errors.sol
	shared/price/Price.sol
	shared/role/Role.sol
	shared/role/RoleStore.sol

Primit-io/primit-contracts-v2-audit

 vault_contract/src/contracts/core/liquidation/LiquidationManager.sol

 vault_contract/src/contracts/core/vault/Vault.sol

 vault_contract/src/contracts/libraries/SignatureVerifier.sol

 vault_contract/src/contracts/referral/IReferralStorage.sol

Finding Categories

Categories	Description
	No description available.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Code Optimization	No description available.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
	No description available.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Primit 2 Security Assessment | CertiK Assessed on Sept 3rd, 2026 | Copyright © CertiK

